

Factory Method Design Pattern in C#

In this article, I will discuss the **Factory Method Design Pattern in C#** with Examples. Please read our previous article discussing the [Factory Design Pattern in C#](#) with one Real-Time Example. The Factory Method Design Pattern belongs to the [Creational Design Pattern](#) Category and is one of Real-Time Applications' most frequently used design patterns.

Note: The most important point is that the Factory Method Design Pattern is not the exact same as the simple Factory Design Pattern we discussed in our previous article. Most people think both are the same; thus, they use the terms Factory and Factory Method interchangeably, which is not right.

What is the Factory Method Design Pattern?

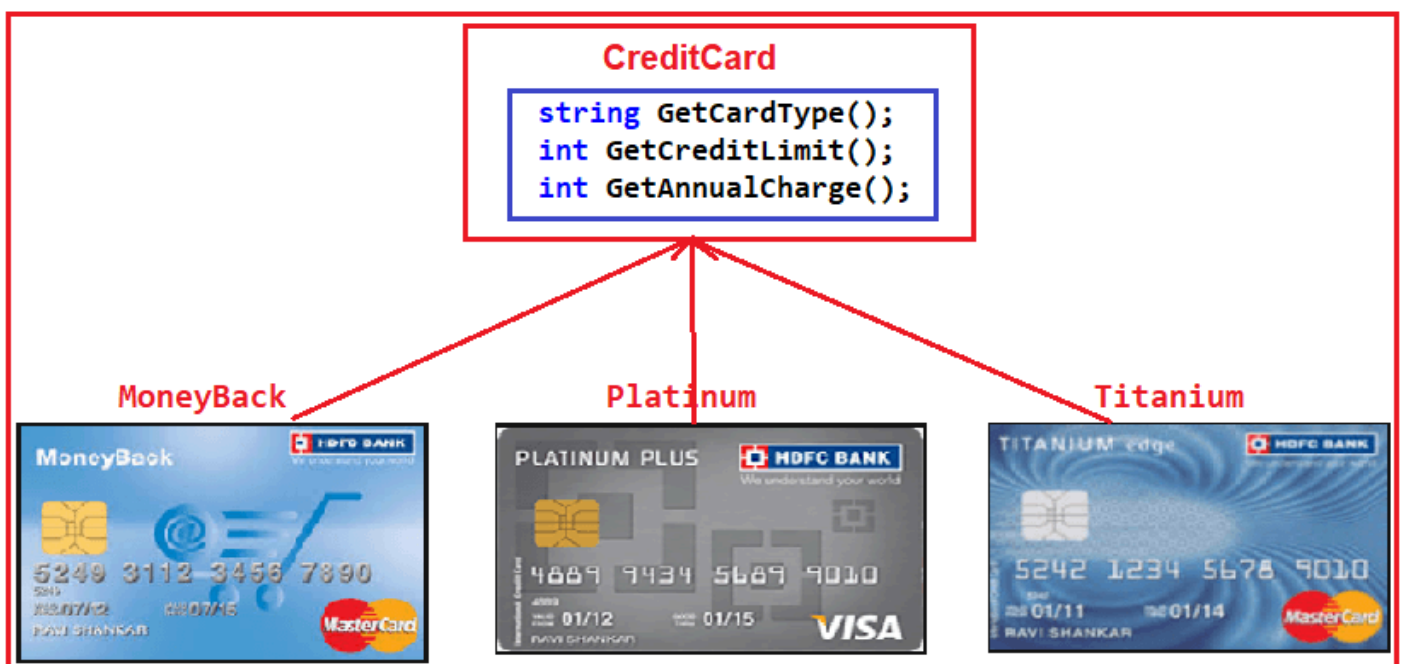
According to the Gang of Four, the Factory Method Design Pattern **Defines an interface for creating an object but lets the subclasses decide which class to instantiate. The Factory method lets a class defer instantiation to subclasses.**

The Factory Method Design Pattern provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created. In the factory method design pattern, we will create an abstract class as the Factory class, which will create and return the product instance, but it will let the subclasses decide which class to instantiate.

Example to Understand Factory Method Design Pattern in C#:

Let us understand the Factory Method Design Pattern with one real-time example using C#. We are going to develop an application to show credit card information. This is the same example we discussed in our previous article, but instead of the Factory Design Pattern, we will use the Factory Method Design Pattern.

As you can see in the diagram below, we have three credit cards, i.e., **MoneyBack**, **Titanium**, and **Platinum**. These credit cards are nothing but our Product classes. Again, these three Credit Card classes are the subclasses of the **CreditCard** super interface (which we can also call Abstract Product). The CreditCard super interface defines the operations (i.e., **GetCardType**, **GetCreditLimit**, and **GetAnnualCharge**) that need to be implemented by the subclasses (i.e., **MoneyBack**, **Titanium**, and **Platinum**), i.e., the Product classes.



As per the definition of the Factory Method Design Pattern, we need to create an abstract class or interface for creating the object. Please have a look at the following diagram. This will be our Abstract Creator Class that declares the Factory Method, i.e., `CreateProduct()`, which will return an object of type Product via the abstract product type, i.e., `CreditCard`. The `CreditCardFactory` Creator class declares the factory method, i.e.,

MakeProduct(), that will return an object of a Product class. The CreditCardFactory subclasses usually provide the implementation of the MakeProduct method.

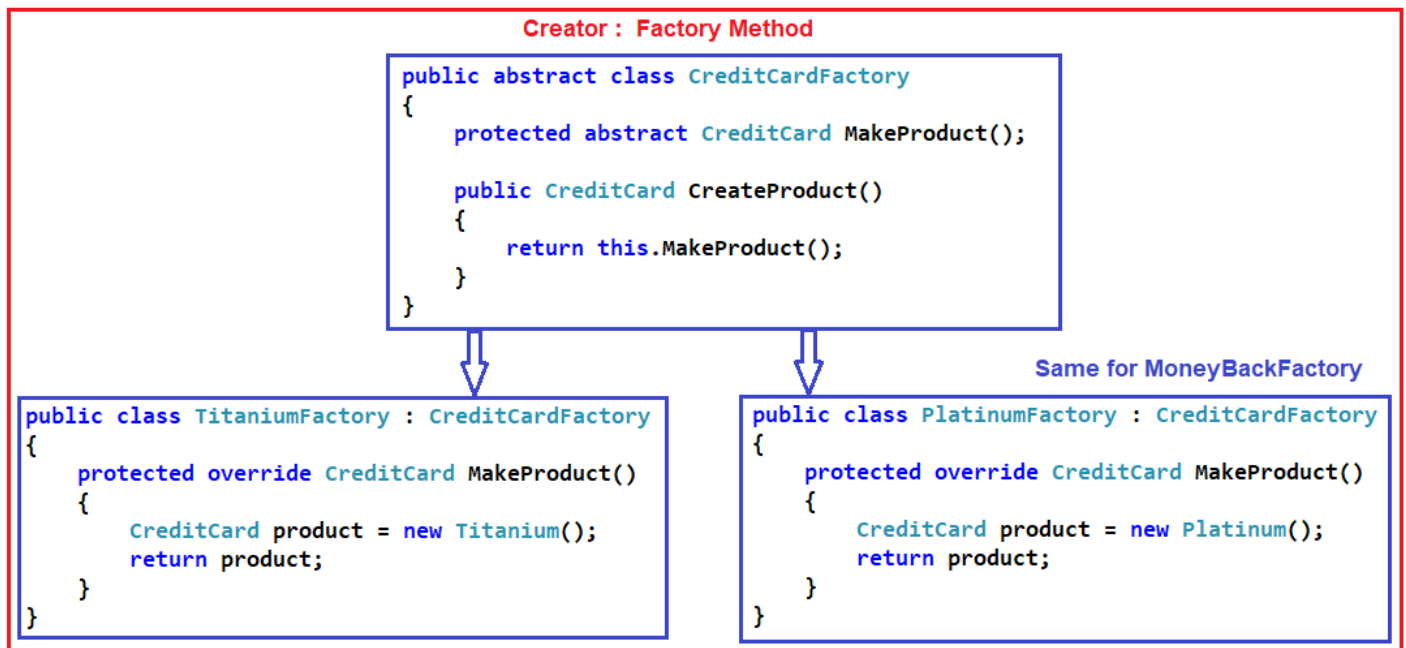
Creator : Factory Method

```
public abstract class CreditCardFactory
{
    protected abstract CreditCard MakeProduct();

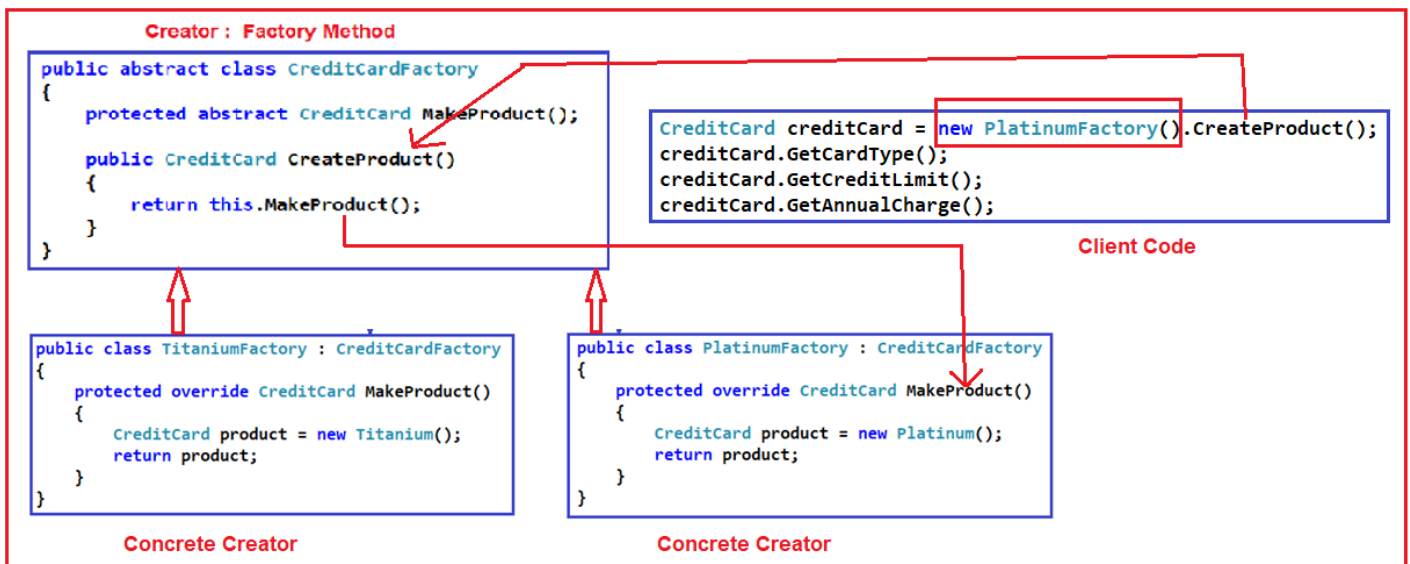
    public CreditCard CreateProduct()
    {
        return this.MakeProduct();
    }
}
```

As you can see, the above abstract class (i.e., CreditCardFactory) contains two methods: one abstract method, i.e., **MakeProduct()**, which is going to be our Factory method, and one concrete method, i.e., **CreateProduct()**. The **CreateProduct()** method internally calls the **MakeProduct()** method of the subclass, which will create and return the product instance.

Please have a look at the following diagram. As we have three credit cards (i.e., MoneyBack, Platinum, and Titanium), we created three subclasses (i.e., PlatinumFactory, TitaniumFactory, and MoneyBackFactory) by implementing the Abstract CreditCardFactory class and providing implementation to the MakeProduct abstract method. This method will return the actual product object, i.e. (MoneyBack, Platinum, and Titanium) via the product interface type, i.e., CreditCard.



Let's see how the client will consume the above CreditCardFactory to create an object. Please have a look at the following diagram. The client code works with an instance of a concrete creator, i.e., the classes that implement the CreditCardFactory abstract class. The CreateProduct method will return the actual product object via the abstract Product interface type, i.e., Creditcard.



The above image shows that the client wants to create a Platinum object. To do so, the clients call the CreateProduct method on the PlatinumFactory instance, which will return a concrete Platinum object via the CreditCard type. Similarly, suppose the client wants to create a Titanium object. In that case, the client needs to call the CreateProduct method on the Titanium Factory instance, which will return a concrete Titanium object.

Implementing the Factory Method Design Pattern in C#:

The Factory Method design pattern involves a structure where a Creator class delegates the responsibility of instantiating its objects to its subclasses. The Factory Method Design Pattern consists of the following components:

- **Product:** This is the interface or abstract class defining the product the factory method will create.
- **ConcreteProduct:** These are the specific implementations of the Product interface or abstract class.
- **Creator:** This abstract class or interface declares the FactoryMethod().
- **ConcreteCreator:** Subclasses of Creator that implement the FactoryMethod() to produce ConcreteProduct instances.

Now, let's proceed to implement the example step by step using the Factory Method Design Pattern in C#.

Step 1: Creating Product Interface (CreditCard)

Create a class file named **CreditCard.cs** and copy and paste the following code. This will be our Product interface, which will provide the signature of the common functionalities that the concrete product classes should implement.

```

namespace FactoryMethodDesignPattern
{
    // The CreditCard interface declares the operations that all
    // concrete products must implement.
    public interface CreditCard
    {
        string GetCardType();
        int GetCreditLimit();
        int GetAnnualCharge();
    }
}
    
```

Step 2: Creating Concrete Products (MoneyBack, Titanium, and Platinum)

This is a class implementing the Product interface (CreditCard) and providing implementation to the interface methods. As we have three types of Credit cards in our example, we will create three classes (**MoneyBack**,

Titanium, and **Platinum**) by implementing the **CreditCard** interface.

MoneyBack.cs

Create a class file named **MoneyBack.cs**, then copy and paste the following code.

```
namespace FactoryMethodDesignPattern
{
    // MoneyBack Product provides implementations of the CreditCard
    interface methods.
    public class MoneyBack : CreditCard
    {
        public string GetCardType()
        {
            return "MoneyBack";
        }
        public int GetCreditLimit()
        {
            return 15000;
        }
        public int GetAnnualCharge()
        {
            return 500;
        }
    }
}
```

Titanium.cs:

Create a class file named **Titanium.cs** and copy and paste the following code.

```
namespace FactoryMethodDesignPattern
{
    //Titanium Product provides implementations of the CreditCard interface
    methods.
    public class Titanium : CreditCard
    {
        public string GetCardType()
        {
            return "Titanium Edge";
        }
        public int GetCreditLimit()
        {
            return 25000;
        }
        public int GetAnnualCharge()
        {
            return 1500;
        }
    }
}
```

Platinum.cs:

Create a class file named **Platinum.cs**, then copy and paste the following code.

```
namespace FactoryMethodDesignPattern
{
    //Platinum Product provides implementations of the CreditCard interface
    methods.
    public class Platinum : CreditCard
    {
        public string GetCardType()
        {
            return "Platinum Plus";
        }
        public int GetCreditLimit()
        {
            return 35000;
        }
        public int GetAnnualCharge()
        {
            return 2000;
        }
    }
}
```

Step 3: Creating Abstract Creator (CreditCardFactory)

The Abstract Creator declares the factory method, which returns an object of type Product. As per the definition, we must create an abstract class or interface to create the object. So, let us create an abstract class that will be our factory class with a publicly exposed method. That method is the factory method, which will return the instance of the product. So, create a class file named **CreditCardFactory.cs** and copy and paste the following code.

```
namespace FactoryMethodDesignPattern
{
    // The CreditCardFactory Creator class declares the factory method
    // that is going to return an object of a Product class.
    // The CreditCardFactory subclasses usually provide the implementation
    of the MakeProduct method.
    public abstract class CreditCardFactory
    {
        protected abstract CreditCard MakeProduct();
        // Also note that The Creator's primary responsibility is not
        creating products.
        // Usually, it contains some core business logic that relies on
        Product objects, returned by the factory method.
        public CreditCard CreateProduct()
        {
            //Call the MakeProduct which will create and return the
            appropriate object
            CreditCard creditCard = this.MakeProduct();
            //Return the Object to the Client
            return creditCard;
        }
    }
}
```

We created the above abstract class with one abstract method, `MakeProduct()`, and one concrete method, `CreateProduct()`. The `CreateProduct()` method internally calls the `MakeProduct()` method of the subclass, which will create and return the product instance. The `MakeProduct()` will be implemented by the concrete creator classes, i.e., the sub-classes of the abstract creator class, `CreditCardFactory`.

Step 4: Creating Concrete Creator (PlatinumFactory, MoneyBackFactory, and TitaniumFactory)

The Concrete Creator class implements the Abstract Creator class and overrides the factory method. The overridden Factory Method will return an instance of a Concrete Product via the base Product interface. As we have three types of Credit Cards, we will create three Concrete Creator classes, namely `PlatinumFactory`, `MoneyBackFactory`, and `TitaniumFactory`, which will implement the abstract Creator Class, namely the `CreditCardFactory` class.

PlatinumFactory Concrete Creator

So, create a class file named **PlatinumFactory.cs** and copy and paste the following code.

```
namespace FactoryMethodDesignPattern
{
    // PlatinumFactory Concrete Creators override the factory MakeProduct
    method
    // to create and return the Platinum Product
    public class PlatinumFactory : CreditCardFactory
    {
        // The signature of the method uses the abstract product CreditCard
        type,
        // Even though the concrete Platinum product is returned from the
        method.
        // This way the Abstract Creator CreditCardFactory can stay
        independent of concrete product classes.
        protected override CreditCard MakeProduct()
        {
            CreditCard product = new Platinum();
            return product;
        }
    }
}
```

MoneyBackFactory Concrete Creator

So, create a class file named **MoneyBackFactory.cs** and copy and paste the following code.

```
namespace FactoryMethodDesignPattern
{
    // MoneyBackFactory Concrete Creators override the factory MakeProduct
    method
    // to create and return the MoneyBack Product
    public class MoneyBackFactory : CreditCardFactory
    {
        // The signature of the method still uses the abstract product
        CreditCard type
        // Even though the concrete MoneyBack product is actually returned
        from the method.
    }
}
```

```

        // This way the Creator can stay independent of concrete product
        classes.
        protected override CreditCard MakeProduct()
        {
            CreditCard product = new MoneyBack();
            return product;
        }
    }
}

```

TitaniumFactory Concrete Creator

So, create a class file named **TitaniumFactory.cs** and copy and paste the following code.

```

namespace FactoryMethodDesignPattern
{
    // TitaniumFactory Concrete Creators override the factory MakeProduct
    method
    // to create and return the Platinum Product
    public class TitaniumFactory : CreditCardFactory
    {
        // The signature of the method uses the abstract product CreditCard
        type,
        // Even though the concrete Titanium product is returned from the
        method.
        // This way the Abstract Creator CreditCardFactory can stay
        independent of concrete product classes.
        protected override CreditCard MakeProduct()
        {
            CreditCard product = new Titanium();
            return product;
        }
    }
}

```

As you can see, the above three classes implement the MakeProduct method of the CreditCardFactory class. That's it. We are done with our implementation. Let's compare the Gang of Four Definition with our example.

According to Gang of Four, we must define an interface or abstract class to create an object. In our example, it is an abstract class, i.e., the **CreditCardFactory** class. The second part of the definition says that let the subclasses decide which class to instantiate. In our example, the subclasses are **PlatinumFactory**, **MoneyBackFactory**, and **TitaniumFactory**. These subclasses will decide which class to instantiate, for example, **MoneyBack**, **Titanium**, and **Platinum**.

Step 5: Consuming the Factory Method in the Client Code:

The client code works with an instance of a concrete creator, i.e., we need to create an instance of either PlatinumFactory, MoneyBackFactory, or TitaniumFactory. Then, we need to call the CreateProduct method on the concrete creator instance, which will return the actual product instance via the product interface, i.e., CreditCard type.

If you want to create an instance of the Platinum CreditCard, call the CreateProduct method on the PlatinumFactory instance. Similarly, if you want the instance of Titanium CreditCard, call the CreateProduct method of the TitaniumFactory instance. So, modify the Main method of the Program class as follows.

```

using System;

```

```

namespace FactoryMethodDesignPattern
{
    class Program
    {
        static void Main(string[] args)
        {
            // The client code works with an instance of a concrete creator
            // The CreateProduct will return the actual product instance
            // via the product interface
            //PlatinumFactory CreateProduct method will return an instance
            // of Platinum Product via the CreditCard interface
            CreditCard creditCard = new PlatinumFactory().CreateProduct();
            if (creditCard != null)
            {
                Console.WriteLine("Card Type : " +
                creditCard.GetCardType());
                Console.WriteLine("Credit Limit : " +
                creditCard.GetCreditLimit());
                Console.WriteLine("Annual Charge : " +
                creditCard.GetAnnualCharge());
            }
            else
            {
                Console.WriteLine("Invalid Card Type");
            }
            Console.WriteLine("-----");
            //MoneyBackFactory CreateProduct method will return an instance
            // of Platinum Product via the CreditCard interface
            creditCard = new MoneyBackFactory().CreateProduct();
            if (creditCard != null)
            {
                Console.WriteLine("Card Type : " +
                creditCard.GetCardType());
                Console.WriteLine("Credit Limit : " +
                creditCard.GetCreditLimit());
                Console.WriteLine("Annual Charge : " +
                creditCard.GetAnnualCharge());
            }
            else
            {
                Console.WriteLine("Invalid Card Type");
            }
            Console.ReadLine();
        }
    }
}

```

When we run the application, it displays the output as expected, as shown in the below image.

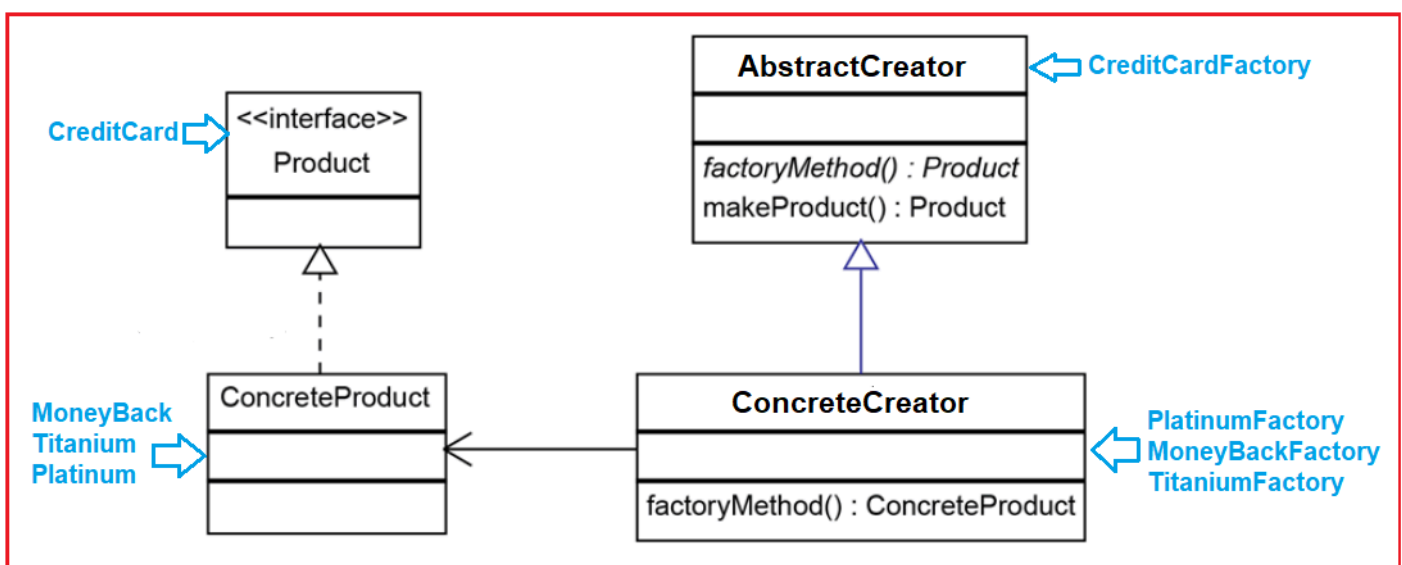
```

Card Type : Platinum Plus
Credit Limit : 35000
Annual Charge :2000
-----
Card Type : MoneyBack
Credit Limit : 15000
Annual Charge :500

```

Factory Method Design Pattern UML (Class) Diagram:

Once we understand how to implement the Factory Method Design Pattern in C#, let us try to understand the UML (Unified Modeling Language) or Class diagram of it. For a better understanding, please look at the following diagram, which shows the different components of the Factory Method Design Pattern. Here, I am comparing the Factory Method Design Pattern UML diagram with our CreditCard example so that you can easily understand the different components of the Factory Method Design Pattern.



Understanding Each Component

Let us understand each component of the Factory Method Design Pattern:

1. **Product:** This is an interface for creating objects. Here, we need to define the Operations a Product should have. In our example, it is the CreditCard interface.
2. **Concrete Product:** This is a class that implements the Product interface. In our example, the MoneyBack, Titanium, and Platinum classes are Concrete Products. These classes implement the Product interface, i.e., CreditCard.
3. **Abstract Creator:** This is an abstract class and declares the factory method, which returns an object of type Product. In our example, the CreditCardFactory abstract class has the CreateProduct Factory method, which will create and return an instance of the actual product. Internally, this CreateProduct method will call the subclass MakeProduct method, creating and returning the appropriate product instance.
4. **Concrete Creator:** These classes implement the Abstract Creator class and override the factory method to return an instance of a Concrete Product.

In the next article, I will discuss [Real-Time Examples of the Factory Method Design Pattern in C#](#). In this article, I explain the **Factory Method Design Pattern in C#** step by step with an example. I hope this article will help you with your needs. I would like to have your feedback. Please post feedback, questions, or comments about this article.